AKADEMIA GÓRNICZO-HUTNICZA

Dokumentacja do projektu

Cinema booking system

z przedmiotu

Języki programowania obiektowego

Elektronika i Telekomunikacja, 3 rok

Mikołaj Polewka

Czwartek, 15:00

prowadzący: Rafał Frączek

15.12.2025r.

1. Opis projektu

Projekt imituje system rezerwacji biletów kinowych umożliwiający użytkownikom przeglądanie aktualnego repertuaru oraz dokonywanie rezerwacji biletów na wybrane seanse filmowe. Użytkownik ma możliwość wyboru konkretnego filmu, godziny seansu, sali kinowej oraz miejsca na sali.

System pozwala również na zarządzanie już istniejącymi rezerwacjami, w tym zmianę wybranego miejsca lub anulowanie biletu w przypadku braku możliwości uczestnictwa w seansie.

Zaimplementowano mechanizmy rejestracji nowych użytkowników oraz logowania do istniejących kont. Dodatkowo w systemie funkcjonuje konto administratora, które posiada rozszerzone uprawnienia umożliwiające zarządzanie salami kinowymi, seansami filmowymi oraz rezerwacjami użytkowników. Funkcjonalność ta pozwala m.in. na czasowe wyłączenie sal z użytku lub anulowanie seansów wraz z przypisanymi do nich rezerwacjami.

2. Project description

The project simulates a cinema ticket reservation system that allows users to browse the current movie repertoire and make ticket reservations for selected film screenings. The user can choose a specific movie, screening time, cinema hall, and seat within the hall.

The system also enables the management of existing reservations, including changing the selected seat or canceling a ticket in case the user is unable to attend the screening.

Mechanisms for user registration and authentication have been implemented, allowing both new and existing users to access the system. Additionally, the system includes an administrator account with extended privileges, enabling the management of cinema halls, movie screenings, and user reservations. This functionality allows, among other things, the temporary exclusion of halls from use or the cancellation of screenings along with all associated reservations.

3. Instrukcja użytkownika

W pierwszym kroku po włączeniu programu mamy do wyboru dwie opcje: zakończenie pracy programu („exit”) lub zalogowanie się na istniejące konto. Jeśli wpisane dane (login + hasło) nie pasują do żadnego konta, możemy zarejestrować nowe konto podając hasło, które będzie chronić nasze konto. Po udanym zalogowaniu lub rejestracji otwiera się nasze menu użytkownika z opcjami przypisanymi pod kolejne cyfry.

- **„1”** - wyświetla aktualnie dostępne w kinie filmy. Po wybraniu numeru konkretnego filmu możemy zobaczyć jego szczegóły lub za klawiszem „0” powrócić do głównego menu. Po obejrzeniu szczegółów wybranego przez nas filmu możemy wrócić do listy filmów by kontynuować przeglądanie („1”) lub wrócić do głównego menu („0”).
- **„2”** – kupowanie biletu na seans. Po wybraniu tej opcji przechodzimy do wyboru filmu, który nas interesuje. Następnie wybieramy seans, na który chcemy się wybrać. Naszym oczom ukazuje się rozkład miejsc w sali kinowej z zaznaczonymi wolnymi (O) i zajętymi (X). Wybieramy najpierw rząd, a następnie kolumnę. Jeśli wybierzemy miejsce, którego nie ma na sali, lub miejsce, które jest zajęte – program nas o tym poinformuje i przeniesie do menu głównego.
- **„3”** – wyświetlenie biletów użytkownika. Wypisuje wszystkie bilety, z informacjami o tytule filmu, godzinie seansu, sali oraz miejscu siedzącym (rząd i kolumna).
- **„4”** – zmiana rezerwacji lub jej wycofanie. Wybieramy konkretny bilet, następnie jedną z dwóch opcji: wycofanie biletu lub zmiana miejsca. W przypadku drugiej opcji wybieramy nowy rząd i kolumnę.

- **„0”** – wyloguj. Ta opcja nie kończy pracy programu, lecz przechodzi do początkowego menu opisanego na wstępie powyższej sekcji.

Każde utworzone przez nas w ramach programu konto ma domyślnie ustawioną rolę **user**. W bazie kont zostało utworzone jedno konto, którego rolą jest **admin**. Pozwala ono na wdrażanie nowości do kina, charakteryzuje się własnym menu, którego opis znajduje się poniżej:

- **„1”** – dodaj salę. Wybieramy numer sali, następnie jej wymiary (rzędy i kolumny). Jeśli wybierzemy numer sali, która już istnieje, zostaniemy o tym poinformowani – sala się nie utworzy.
- **„2”** – usuń salę. Wybieramy numer sali, którą chcemy usunąć z systemu. Wraz z usuniętą salą zostają usunięte również seanse, które są w niej zaplanowane, oraz bilety z nimi powiązane.
- **„3”** – dodaj seans. Wybieramy film, a następnie salę, w której będzie rozegrany (dostępne sale zostaną wyświetlone). Na końcu wpisujemy godzinę seansu (format godzina:minuta).
- **„4”** – usuń seans. Program wyświetla listę wszystkich seansów w kinie (tytuł | godzina | sala). Wybieramy numer seansu, który chcemy usunąć.
- **„5”** – usuń rezerwację użytkownika. Po wybraniu tej opcji wyświetlane są wszystkie rezerwacje (tytuł | godzina | sala | rząd | miejsce). Wybieramy numer biletu, który chcemy usunąć.
- **„0”** – analogicznie do panelu użytkownika – wylogowanie.

Admin **nie może** dodać nowego filmu do kina. Nowe filmy zostają podsyłane przez dystrybutorów i umieszczane w bazie filmów. Tylko filmy w bazie filmów mogą zostać pokazane w kinie.

4. Kompilacja

Projekt został przygotowany w języku C++ i korzysta ze standardowych bibliotek STL. Do poprawnego działania wymaga kompilatora C++ obsługującego standard C++11 lub nowszy.

Do kompilacji w systemie Windows można użyć wbudowanego terminala VS Code i polecenia:

g++ src/*.cpp -linclude -o kino

Po wpisaniu komendy **./kino** następuje uruchomienie programu. Wszystkie ścieżki w kodzie zostały przygotowane tak, aby powyższa kompilacja działała również w systemie Linux. Jedynym warunkiem przejścia kompilacji jest wpisanie komendy w odpowiednim miejscu projektu – w tym samym folderze, w którym znajdują się 3 główne foldery projektu.

5. Pliki źródłowe

Projekt składa się z następujących plików źródłowych:

- *LoginSystem.h, LoginSystem.cpp* – deklaracja i implementacja klasy odpowiedzialnej za logowanie oraz rejestrację użytkowników.
- *Movie.h, Movie.cpp* – deklaracja i implementacja klasy reprezentującej film (tytuł, czas trwania, opis, kategoria).
- *Hall.h, Hall.cpp* – deklaracja i implementacja klasy reprezentującej salę kinową (numer, liczba rzędów i kolumn) oraz operacje na salach (dodawanie, zapis do pliku).
- *Screening.h, Screening.cpp* – deklaracja i implementacja klasy seansu (film, godzina, sala, dostępne miejsca) oraz metody do wyświetlania i rezerwacji miejsc.

- *Reservation.h*, *Reservation.cpp* – deklaracja i implementacja klasy reprezentującej rezerwację użytkownika (login, film, czas, sala, rząd, miejsce) oraz operacje na rezerwacjach (zapis/odczyt z pliku).
- *AdminManager.h*, *AdminManager.cpp* – deklaracja i implementacja klasy zarządzającej panelem administratora (dodawanie/usuwanie sal i seansów, zarządzanie rezerwacjami).
- *UserManager.h*, *UserManager.cpp* – deklaracja i implementacja klasy zarządzającej panelem zwykłego użytkownika (przeglądanie filmów, kupowanie i modyfikacja biletów).
- *main.cpp* – główny plik programu, odpowiedzialny za inicjalizację systemu, logowanie użytkowników oraz uruchomienie odpowiednich menedżerów (Admin/User).

6. Zależności

Brak.

7. Opis klas

W projekcie utworzono następujące klasy:

➢ **LoginSystem** – obsługuje logowanie i rejestrację użytkowników.

- *void loadAccounts()* – wczytuje istniejące konta z pliku.
- *std::string login(const std::string& login)* – loguje użytkownika, zwraca jego rolę (user/admin).
- *void registerUser(const std::string& login)* – rejestruje nowego użytkownika i zapisuje dane do pliku.

➢ **Movie** – reprezentuje film w systemie.

- *std::string getTitle() const* – zwraca tytuł filmu.
- *std::string getCategory() const* – zwraca kategorię filmu.
- *int getDuration() const* – zwraca czas trwania filmu w minutach.
- *std::string getDescription() const* – zwraca opis filmu.
- *static std::vector<Movie> loadFromFile(const std::string& path)* – wczytuje filmy z pliku.
- *static void displayMovieList(const std::vector<Movie>& movies)* – wyświetla listę filmów.

➢ **Hall** – reprezentuje salę kinową.

- *int getNumber() const* – numer sali.
- *int getRows() const, int getCols() const* – liczba rzędów i kolumn w sali.
- *static std::vector<Hall> loadFromFile(const std::string& path)* – wczytuje listę sal z pliku.
- *static void saveToFile(const std::vector<Hall>& halls, const std::string& path*) – zapisuje listę sal do pliku.

➢ **Screening** – reprezentuje seans filmu w konkretnej sali i czasie.

- *Movie getMovie() const* – zwraca film przypisany do seansu.
- *std::string getTime() const* – zwraca godzinę seansu.
- *int getHallNumber() const* – zwraca numer sali.

- *void display() const* – wyświetla informacje o seansie.
- *void displaySeats() const* – wyświetla układ miejsc w sali.
- *bool reserveSeat(int row, int col)* – rezerwuje miejsce w sali.
- *void applyExistingReservations(const std::vector<Reservation>& reservations)* – zaznacza miejsca już zarezerwowane.
- *int getAvailableSeats() const* – zwraca liczbę wolnych miejsc.
- *static std::vector<Screening> loadFromFile(const std::string& path, const std::vector<Movie>& movies)* – wczytuje seanse z pliku.
- *static void saveAll(const std::vector<Screening>& screenings, const std::string& path)* – zapisuje seanse do pliku.
- *static void displayScreenings(const std::vector<Screening>& screenings)* – wyświetla listę seansów.

➢ **Reservation** – reprezentuje rezerwację użytkownika.

- *const std::string& getLogin() const* – login użytkownika.
- *const std::string& getMovieTitle() const* – tytuł filmu.
- *const std::string& getTime() const* – godzina seansu.
- *int getHall() const, int getRow() const, int getCol() const* – numer sali, rząd i kolumna miejsca.
- *static std::vector<Reservation> loadFromFile(const std::string& path)* – wczytuje rezerwacje z pliku.
- *static std::vector<Reservation> loadUserReservations(const std::string& login, const std::string& path)* – wczytuje rezerwacje danego użytkownika.
- *static void saveAllReservations(const std::vector<Reservation>& reservations, const std::string& path)* – zapisuje wszystkie rezerwacje do pliku.
- *static void displayReservations(const std::vector<Reservation>& reservations)* – wyświetla listę rezerwacji.

➢ **AdminManager** – zarządza panelem administratora.

- *void run()* – główne menu administratora.
- *void addHall()* – dodaje salę.
- *void removeHall()* – usuwa salę wraz z seansami i rezerwacjami.
- *void addScreening()* – dodaje seans w wybranej sali.
- *void removeScreening()* – usuwa seans i powiązane rezerwacje.
- *void removeReservation()* – usuwa rezerwację użytkownika.

➢ **UserManager** – zarządza panelem zwykłego użytkownika.

- *void run()* – główne menu użytkownika.
- *static void clearScreen()* – czyszczenie ekranu.
- Metody pozwalają na przeglądanie filmów, wybór seansu, rezerwację miejsca, wyświetlenie własnych biletów, zmianę lub anulowanie rezerwacji.

8. Zasoby

W projekcie wykorzystywane są następujące pliki zasobów (wszystkie w folderze database):

- `konta.txt` – przechowuje dane użytkownika. Struktura pliku (wszystko w jednej linii, oddzielone spacją):
  - login użytkownika
  - hasło
  - rola

  W następnych linijkach dane pozostałych klientów
- `filmy.txt` – przechowuje informacje o filmach. Struktura pliku (wszystko w jednej linii, oddzielone średnikami):
  - tytuł filmu
  - kategoria
  - czas trwania w minutach
  - opis

  W następnych linijkach dane pozostałych filmów.
- `sale.txt` – przechowuje listę sal kinowych. Struktura pliku (każda linijka, oddzielone spacjami):
  - numer sali
  - liczba rzędów
  - liczba kolumn

  W następnych linijkach dane pozostałych sal.
- `seanse.txt` – przechowuje informacje o seansach. Struktura pliku (każda linijka, oddzielone średnikami):
  - tytuł filmu
  - godzina seansu
  - numer sali
- `rezerwacje.txt` – przechowuje rezerwacje użytkowników. Struktura pliku (każda linijka, oddzielone średnikami):
  - login użytkownika
  - tytuł filmu
  - godzina
  - numer sali
  - rząd
  - kolumna

9. Dalszy rozwój i ulepszenia

Projekt można w przyszłości rozbudować o następujące funkcjonalności:

- Wydruk biletów: możliwość generowania biletów na konkretne seanse, wraz z zawarciem na jednym bilecie wszystkich miejsc zarezerwowanych na konkretny seans (obecnie przy oglądaniu biletów są one wszystkie oddzielnie).
- Interfejs graficzny
- Integracja z bazą danych: zamiast plików tekstowych można by użyć bazy danych (np. SQLite lub MySQL) dla większej wydajności, skalowalności i bezpieczeństwa danych.

10. Inne

Brak.